Table of Contents

Table Of Figures ***2***

Acknowledgement ***3***

Introduction ***4***

L01: Examine the key components related to the object–oriented–programming paradigm, analyzing design pattern types. ***6***

Part 1.1: Characteristics of Object – Orientated Paradigm **6**

Types of Programming Paradigms 6

Object - Oriented Paradigms 6

Encapsulation 12

Polymorphism 14

Abstraction 16

Class and Objects 17

Aggregation 18

..... **18**

Composition 18

Generalization 19

Associative 19

Part 1.2: Report on Design Patterns **20**

Part 1.3: Relationship Between Object Oriented Paradigm and Design Patterns. **41**

L02: Design a Series of UML class Diagrams ***43***

Part 2.1: Design A Class Diagram for The Scenario. **43**

Part 2.2: Discuss What Type of Test Driven Development **47**

Advantages 48

Disadvantages 48

Tools 48

L03: Implement Code Applying Design Patterns ***50***

Part 3.1: Develop Code That Implements Design Pattern. **50**

Part 3.2: Test Case for The Source Code. **51**

L04: Investigate scenarios with Respect to Design Patterns. ***53***

Part 4.1: Design Pattern for The Scenario **53**

Conclusion ***54***

References ***55***

Table Of Figures

Figure 1:Example diagram of a hierarchical Inheritance 11
Figure 2:Example of aggregation.....18
Figure 3:Example diagram of Composition.....18
Figure 4:Example of diagram Generalization 19
Figure 5:Examples diagram of Associative.....19
Figure 6:Class Diagram of Singleton Design Pattern 22
Figure 7:Class Diagram of Factory pattern to Given Code.....25
Figure 8:Example of Adapter Pattern 26
Figure 9:Class Diagram of Adapter Pattern.....28
Figure 10:Example for Decorator Pattern 29
Figure 11:Class Diagram of Decorator Pattern.....32
Figure 12:Iterator Pattern Example.....34
Figure 13:Class Diagram of Iterator Pattern 36
Figure 14:Example of Template method pattern 37
Figure 15:Class for the Given Template method pattern.....40
Figure 16:UML diaram attributes and Operations.....43
Figure 17:Symbols of UML diagram 44
Figure 18:Line notations for UML diagrams.....44
Figure 19:UML diagram of Given Book Store Scenario.....45
Figure 20: Test Driven Development Process 47

Acknowledgement

The success and final outcome of this assignment required a lot of guidance and assistance from many people and I am extremely fortunate to have got this all along the completion of my assignment work. Whatever I have done is only due to such guidance and assistance and I would not forget to thank them.

I respect and thank Mr. B. Sabashan for giving me an opportunity to do this assignment on time, I extremely grateful to him for providing such a nice support and guidance.

I am really grateful because I managed to complete this assignment within the time given by my lecturer. This assignment cannot be completed without the effort and co-operation from my class mates. I would like to express my gratitude to my friends and respondents for support and willingness to spend some time with me.

A. Nirojan

Introduction

Advance programming is a subject that contain all programming language paradigms and Object-oriented programming is a programming language model organized around the objects rather than “action” and data rather than logic. Historically, a program has been viewed as a logical procedure that takes input data, processes it, and produces output data.

In this assignment I have required to develop software system for a Book Store before that I have to understand the Advance Programming principles and able to designing the object-oriented programming solution.

Implement the solution, designing the algorithm. Finally, I have maintain the source code in the GitLab and have a report as well as testing and maintenance plan for the developed system.

L01

L01: Examine the key components related to the object–oriented–programming paradigm, analyzing design pattern types.

Part 1.1: Characteristics of Object – Orientated Paradigm

The programming paradigm is the general type approach to programming or solution of problems using the programming language.

Types of Programming Paradigms

- Procedural Paradigm
- Object-oriented Paradigm
- Functional Paradigm
- Logical Paradigm

Object - Oriented Paradigms

The object-oriented paradigm is the process to address all the issues, boundaries between phases are blurred in the SDLC (Software Development Lifecycle) because the objects are the items of interest in all phases.

This is a method of programming that involves the creation of the intellectual's object that model a business problem that we will try to solve.

In the traditional software development is not support iteration, reuse or unifying model to integrate phases of the program lifecycle. It is the next logical step in a progression that led from purely procedural approach to an object-based approach and to the object-oriented approach.

Problems in Object Oriented Programming Is From:

- Poor Design and Analysis
- Poor Maintenance

The object-oriented programming paradigm helps to overcome these problems by analysis and designing phases in the initialization software development phase.

Characteristics of Object-Oriented Paradigm

- Inheritance
- Encapsulation
- Polymorphism
- Abstraction
- Objects
- Class

Inheritance

The inheritance is concept of where the properties of the parent class inherited to child class. It helps to reuse the codes in a good manner and used to establish relationship between different classes.

Eg:
If we take the father and son the Father is a base class or super class and son is the child class or derive class.

Some Types

- Single Level Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance

Single Inheritance

The single inheritance means one class inherits their properties to another class that means child class derives the properties from parent class or super class.

```
package bcas.inher.sc;

public class Company {

	int workerId;
	String workerName;
	//String shiftTime;

	public Company(int workerId, String workerName) {
		this.workerId = workerId;
		this.workerName = workerName;
	}

	public void workD() {
		System.out.println("Your Worker Id :" + workerId);
		System.out.println("Your Worker Name :" + workerName);
		//System.out.println("Your Worker Name :" +shiftTime);
	}
}
```

- In this class the Company is the parent or super class and the Depart is the child class and it extends from Company means it derives properties from parent class that is Company class.

```
package bcas.inher.sc;

public class Depart extends Company {
	String departmentName;
	String shiftTime;
	String address;

	public Depart(int workerId, String workerName, String departmentName, String workTime, String address) {
		super(workerId, workerName);
		this.departmentName = departmentName;
		this.shiftTime = workTime;
		this.address = address;

	}
}
```

- The is the Execution class of This Single Inheritance contain main method.

```
package bcas.inher.sc;

public class Demo {
	public static void main(String[] args) {
		Depart work = new Depart(001, "Nirojan", "Testing", "Morning", "Jaffna");
		Depart wor = new Depart(002,"Anton","Developing","Afternoon","Chunnakam");
		work.workD();
		wor.workD();
	}
}
```

Multilevel Inheritance

The Class can be derived from another derived class is called as Multilevel Inheritance. It will take the derived class as the base for new class.

- This class is parent class Vehicle.

```
public class Vehicle {

              public Vehicle()
              {
                 System.out.println("Class Vehicle");
              }
              public void vehicleType()
              {
                 System.out.println("Vehicle");
              }

}
```

- In This class Car Derived from Parent class Vehicle.

```
public class Car extends Vehicle{

              public Car()
              {
                 System.out.println("Class Car");
              }
              public void vehicleType()
              {
                 System.out.println("Vehicle Type: Car");
              }
           }
```

- In This class Maruthi class derived from the derived class Car. This is the easiest example of multilevel Inheritance.

```
public class Maruthi extends Car {

      public Maruthi() {
            System.out.println("Class Maruti");
      }

      public void brand() {
            System.out.println("Brand: Maruti");
      }

      public static void main(String args[]) {
            Maruthi obj = new Maruthi();
            obj.vehicleType();
            obj.brand();
      }
}
```

Output for This Code:

```
Class Vehicle
Class Car
Class Maruti
Vehicle Type: Car
Brand: Maruti
```

Hierarchical Inheritance

In this type of inheritance one class can be inherited by many sub classes. This means if we have a parent class named as **Class Bird** and it will be inherited by **Class Parrot, Class Dove, Class Owl** and **Class Woodpecker.**

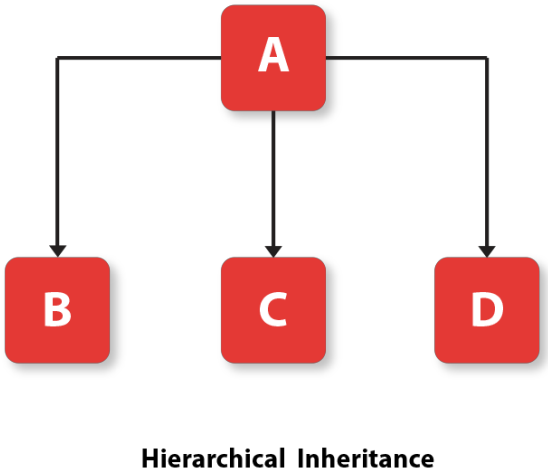

Figure 1:Example diagram of a hierarchical Inheritance

Encapsulation

Encapsulation is one of the fundamental in OOP concept and in java it is a mechanism of wrapping the data variables and code acting on the data methods together as a single unit. It helps to create programs that are easier to maintain and debug.

Encapsulation is to declare variables with different access modifiers, such as private or public. However, when using private variables in programming languages such as java. Declare the variables of class as private, provide public setter and getter methods to modify and view the variable values these are the ways to achieve encapsulation in java.
If we are using Encapsulation it will able to give

- Fields of class made read and write only
- Class will have control over what is stored in their fields.

Example Code For Encapsulation

```
package bcas.encap;
public class Encapsulation {

	private String carType;
	private String model;
	private int year;

	public int getYear() {
		return year;
	}

	public String getCarType() {
		return carType;
	}

	public String model() {
		return model;
	}

	public void setYear(int newYear) {
		year = newYear;
	}

	public void setCarType(String newCarType) {
		carType = newCarType;
	}

	public void setModel(String newModel) {
		model = newModel;
	}
}
```

- Main Class

```
package bcas.encap;

public class EncapsulationDemo {

     public static void main(String args[]) {
          Encapsulation encap = new Encapsulation();
          encap.setCarType("Sedan");
          encap.setYear(2018);
          encap.setModel("BMW");
          System.out.println("Car Type Is : " +
encap.getCarType() + " Released Year Is : " +
encap.getYear());
     }
}
```

Polymorphism

Polymorphism has an ability to use same code with different types of objects and behave differently with each. If we have a problem that is change behavior depending on concrete objects and the solution for that is polymorphism.

Polymorphism allows to define one interface and have many implementations.

Method Overriding. – Two methods with same method name and parameters and overriding allows a child class to provide a specific implementation of a method that is already provided its parent class.

Method Overloading - *Overloading* occurs when two or more methods in one class have the same method name but different parameters.

Types of Polymorphism

- Run Time
- Compile Time

Method Overriding is example for run time and Method Overloading is example for Compile Time.

- **Example for polymorphism**

```
package bcas.poly;

public class Human {
	public void type() {
		System.out.println("Humans Are Two Types");
	}
}
```

- Child Class with same type Method

```
package bcas.poly;

public class Man extends Human {
	@Override
	public void type() {
		System.out.println("Man");
	}

	public static void main(String args[]) {
		Human types = new Man();
		types.type();
	}
}
```

Abstraction

It is referred to ad quality of dealing with ideas that events and it is basically dealing with hiding details and show the essential thing to the user.

We will achieve abstraction in two ways and they are Abstract Class and Interface.

Abstract classes contain abstract keyword and if a class declared as abstract then cannot create an object of an abstract class and it can contain abstract methods and concrete methods.

A normal cannot have abstract methods.

```
Abstract class SmartDevice {

Abstract void Phone() {
        }
}
```

Interface is a duplicate of a class or collection of abstract methods and static constants. In interfaces each methods public and abstract but it do not contain any constructor.

- Interface Class

```
package bcas.abstraction;

public interface Vehicle {

     public void wheels(int many);

     public void applyBrakes(int decrease);
}
```

```
package bcas.abstraction;

public class MotorBike implements Vehicle{

            int wheels=2;
            int applyBrakes=1;
            @Override
            public void wheels(int many) {

            }
            public void applyBrakes(int decrement)
            {
            applyBrakes=decrement;
            }
            void printStates(){
                        System.out.println("Wheels is :"+ wheels +
            "Did Apply Brake:"+applyBrakes);
            }
            public static void main(String[] args) {
            // TODO Auto-generated method stub
            MotorBike bike= new MotorBike();

            bike.wheels(50);
            bike.applyBrakes(3);
            bike.printStates();
            bike.printStates();

            }
}
```

Class and Objects

Classes create objects and objects use methods to communicate with them. A class is a collection of fields and methods that operate on the data.

Example of Class:

```
                  public class Dog {

}
```

Example of methods:

```
                        Public int minimum(){

                                    }
```

Aggregation

Aggregation is a special case of association and a direct association between objects. If an object has another object then it is an aggregation between it.

It is representing by Has-A relationship and in this both entries can survive them individually it means ending one entity will not affect other entity.

We use aggregation because code reuse is a best achievement by aggregation.

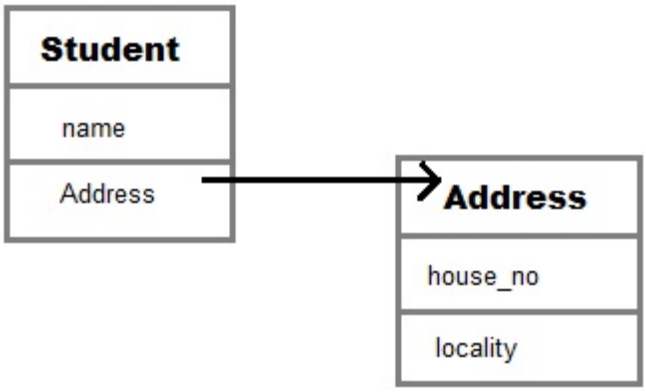

Figure 2:Example of aggregation

Composition

Composition is a special case of aggregation that is a restricted aggregation is called composition. If an object contains another object and contained object cannot be exist without existence of container object.

It represents part-of relationship and in this both entities are depending on each other.

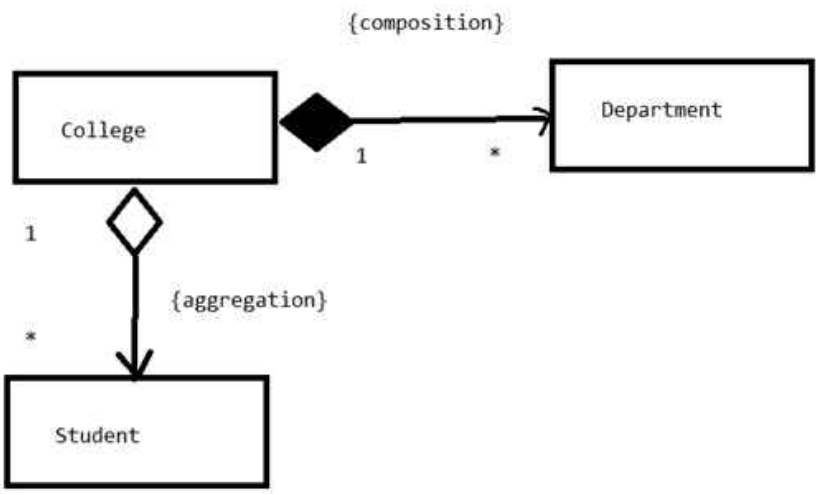

Figure 3:Example diagram of Composition

Generalization

It is a process of exacting shared characteristics from two or more classes and combining them into generalized super class. Generalization is also called a “Is A Relationship”.

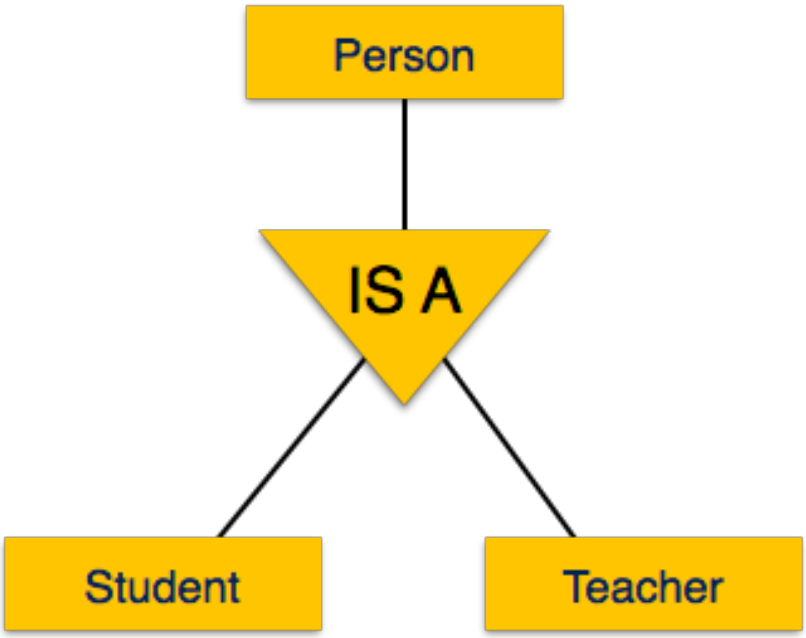

Figure 4:Example of diagram Generalization

Associative

It is a relationship between two objects and it defines the multiplicity between objects.

One to One, Many to One, Many to Many, Many to One all these relationships are association between objects.

Aggregation is a special form of association and composition is a special form of aggregation.

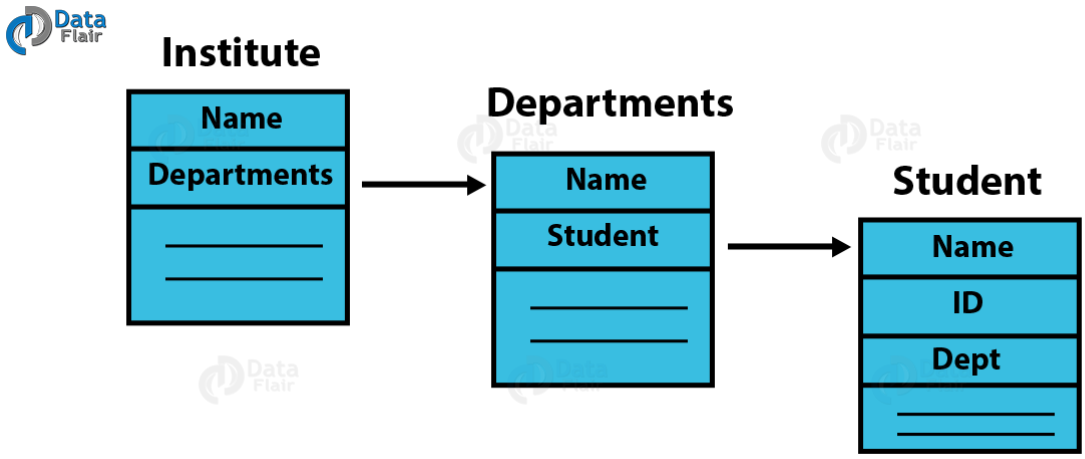

Figure 5:Examples diagram of Associative

Part 1.2: Report on Design Patterns

The code will be flexible we have enough time to make any changes in less time and our code will be reusable in anywhere with any trouble. We will maintain our code easily. If we use the design patterns it will saves a lot of time.

Why we use Design Patterns

- **Reusability –** Coupled and cohesive objects and classes will make code more reusable. It will be easy to test when compare to highly coupled codes.

- **Flexibility -** Using design patterns your code becomes flexible. It helps to provide the correct level of abstraction due to which objects become loosely coupled to each other which makes your code easy to change.

Types of Design Patterns

- **Creational Patterns**
- **Structural Patterns**
- **Behavioral Patterns**

Creational Patterns

Creational design patterns are used to design the instantiation process of objects. The creational pattern uses the inheritance to vary the object creation. Creation object leads to unwanted complexity in design problems. Creational patterns provide help or solution by control creation of objects in many ways.

Singleton Pattern - Singleton patterns restrict the initiation of class and ensure that one instance of class exist on the java virtual machines.

The difference between normal class and singleton class is if we take a constructor for normal class, for singleton class we will take getInstance () method. To design a singleton class, we need to make constructor as private and write static method that returns type of object of singleton class.

- Singleton pattern class

```
package bcas.singleton;

public class Singleton {
        private static Singleton single_instance = null;
        public String s;

        private Singleton()
        {
            s = "Hello I am Singleton ";
        }

        public static Singleton getInstance()
        {
            if (single_instance == null)
                single_instance = new Singleton();

            return single_instance;
        }
    }
```

- Class Contain main method and calling class

```
package bcas.single;

import bcas.singleton.Singleton;

public class Main {
        public static void main(String args[])
        {
            Singleton x = Singleton.getInstance();
            Singleton y = Singleton.getInstance();
            Singleton z = Singleton.getInstance();


            x.s = (x.s).toUpperCase();
```

```
            System.out.println("Hi Guys " + x.s);
            System.out.println("String Class " + y.s);
            System.out.println("String And " + z.s);
            System.out.println("\n");


            z.s = (z.s).toLowerCase();

            System.out.println("String from x is " + x.s);
            System.out.println("String from y is " + y.s);
            System.out.println("String from z is " + z.s);
        }
    }
```

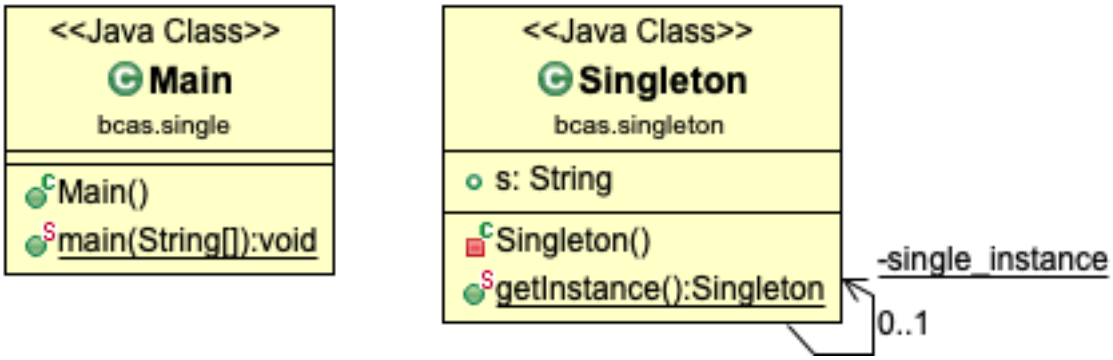

Figure 6:Class Diagram of Singleton Design Pattern

Output Of This Code:

```
Hi Guys HELLO I AM SINGLETON
String Class HELLO I AM SINGLETON
String And HELLO I AM SINGLETON


String from x is hello i am singleton
String from y is hello i am singleton
String from z is hello i am singleton
```

Factory Pattern

Factory pattern is one of the main design patterns in java world. This is one of the creational patterns which gives best way to create an object.

Factory methods leads to create parallel class hierarchies and give ways to encapsulate the instantiation appropriate class inside designated method referred to as factory method.

Advantages

- Factory design pattern provides approach to code for interface rather than implementation.
- This pattern removes instantiation of actual implementation classes from client code,
- Provides abstraction between implementation and client classes through inheritance.

When the implementation of an interface or abstract class expect to change the frequently, the current implementation cannot comfortable accommodate new change, Initialization process relatively simple and constructor only requires a handful of parameters and in this situation, we want to use factory method to implement the codes.

- **Interface of Class Dog with method speak**

```
package creational.factory;

public interface Dog {

	public void speak();
}
```

- **Implementation of class Dog in Poodle class**

```
package creational.factory;

public class Poodle implements Dog {

	public void speak() {
		System.out.println("The poodle says \"arf\"");
	}
}
class Rottweiler implements Dog {
	public void speak() {
		System.out.println("The Rottweiler says (in a very
deep voice) \"WOOF!\"");
```

```
    }
}
```

Class Test Dog

```
package creational.factory;

public class TestDog {

    public static Dog getDog(String criteria) {

        if (criteria.equals("small"))
            return new Poodle();
        else if (criteria.equals("big"))
            return new Rottweiler();

        return null;
    }
}
```

Class Factory Dog

```
package creational.factory;

import creational.factory.TestDog;

public class FactoryDog {

public static Dog getDog(String criteria) {

        if (criteria.equals("small"))
            return new Poodle();
        else if (criteria.equals("big"))
            return new Rottweiler();

        return null;
    }

      public static void main(String[] args)
      {
        // create a small dog
        Dog dog = FactoryDog.getDog("small");
        dog.speak();

        // create a big dog
        dog = FactoryDog.getDog("big");
        dog.speak();
      }
    }
```

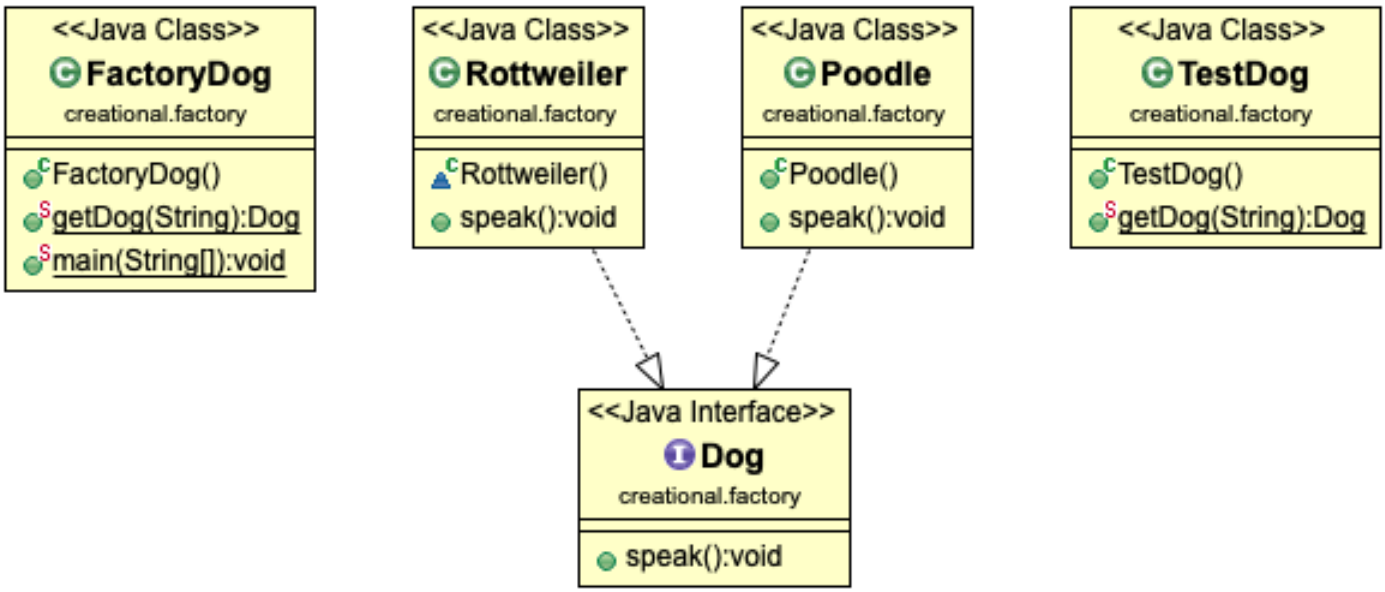

Figure 7:Class Diagram of Factory pattern to Given Code

Output for This Code:

```
The poodle says "arf"
The Rottweiler says (in a very deep voice) "WOOF!"
```

Structural Patterns

Structural pattern gives many ways to create class structures. They are concerned with how classes and objects compose to form larger structures and they uses inheritance to compose interfaces or implementation.

Multiple inheritance can be used to combine features from one or more classes into single class. This types pattern uses for make independently developed class libraries work together.

Types

- Adapter Pattern
- Façade Pattern
- Decorator Pattern
- Bridge Pattern

Adapter Pattern

Adapter patterns let to adapt what an object or class exposed and what other class or object expects. It converts interface of class into another interface that the client expects.

- **When To Use Adapter Pattern –** Having a class and its interface does not match the one that need, classes that don't compatible with interfaces and in this time we will use adapter pattern implementation.

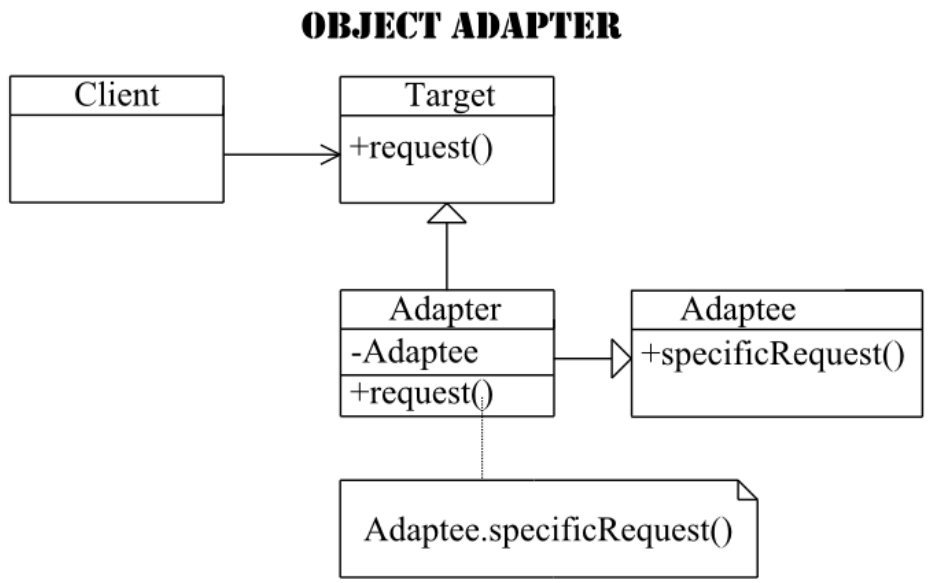

Figure 8:Example of Adapter Pattern

- **Class Bird**

```
package structural.adapter;

public interface Bird {

     public void fly();

     public void makeSound();
}
```

- **Class Sparrow**

```
package structural.adapter;

public class Sparrow implements Bird {
          public void fly()
          {
               System.out.println("Flying");
          }
          public void makeSound()
          {
               System.out.println("Chirp Chirp");
          }
     }
```

- **Class MainM**

```
package structural.adapter;

public class MainM {
          public static void main(String args[])
          {
               Sparrow sparrow = new Sparrow();
               System.out.println("Sparrow...");
               sparrow.fly();
               sparrow.makeSound();

               System.out.println("ToyDuck...");
          }
     }
```

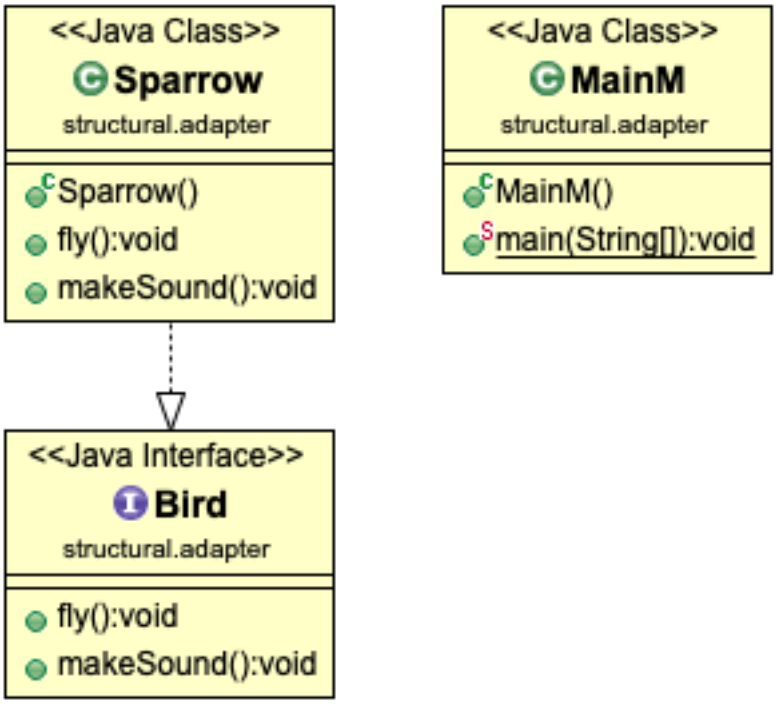

Figure 9:Class Diagram of Adapter Pattern

Output for This Code:

```
Sparrow...
Flying
Chirp Chirp
ToyDuck..
```

Decorator Pattern

Decorator pattern provide alternative to sub classing for extending the functionality and is used to extend functionality of an object dynamically without having change original class using inheritance.

The Decorator object is designed to have same interface as the underlying object. Decorator prevents proliferation of subclasses leasing to less complexity and confusion.

Decorator pattern in following cases:

- add responsibilities to individual objects dynamically and transparently without affecting other objects.
- For responsibilities that can be withdrawn.
- When extension by sub-classing is impractical. Sometimes a large number of independent extensions are possible and would produce an explosion of subclasses to support every combination. Or a class definition may be hidden or otherwise unavailable for sub-classing.

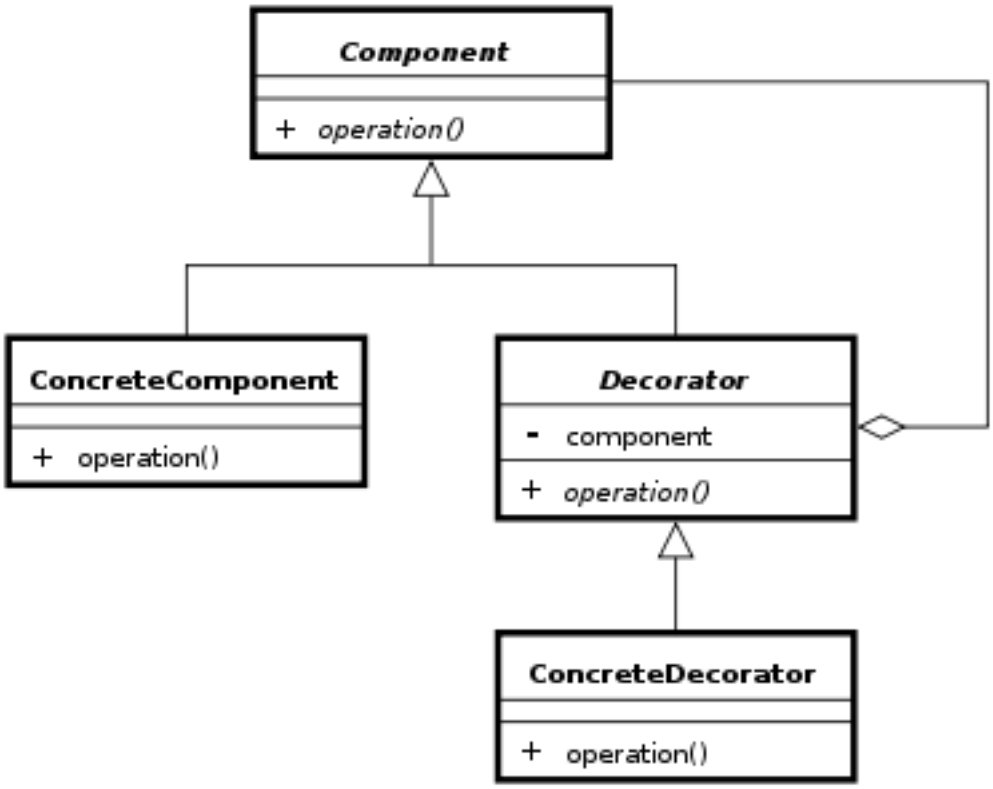

Figure 10:Example for Decorator Pattern

Interface I

```
package structural.decorator;

public interface I {
        void doIt();
    }
```

Class A implements Interface I

```
package structural.decorator;

public class A implements I {
        public void doIt() {
            System.out.print('A');
        }
    }
```

Class D Implements Interface I

```
package structural.decorator;

abstract class D implements I {
        private I core;
        public D(I inner) {
            core = inner;
        }

        public void doIt() {
            core.doIt();
        }
    }
```

Class X extends D

```
package structural.decorator;

public class X extends D {
        public X(I inner) {
            super(inner);
        }

        public void doIt() {
            super.doIt();
            doX();
        }

        private void doX() {
            System.out.print('X');
        }
    }
```

MainDemo class contain Main Method

```
package structural.decorator;

public class MainDemo {

        public static void main( String[] args ) {
            I[] array = {new X(new A()), new X(new X(new
A())), new X(new A())};
            for (I anArray : array) {
                anArray.doIt();
                System.out.print("  ");
            }
        }
    }
```

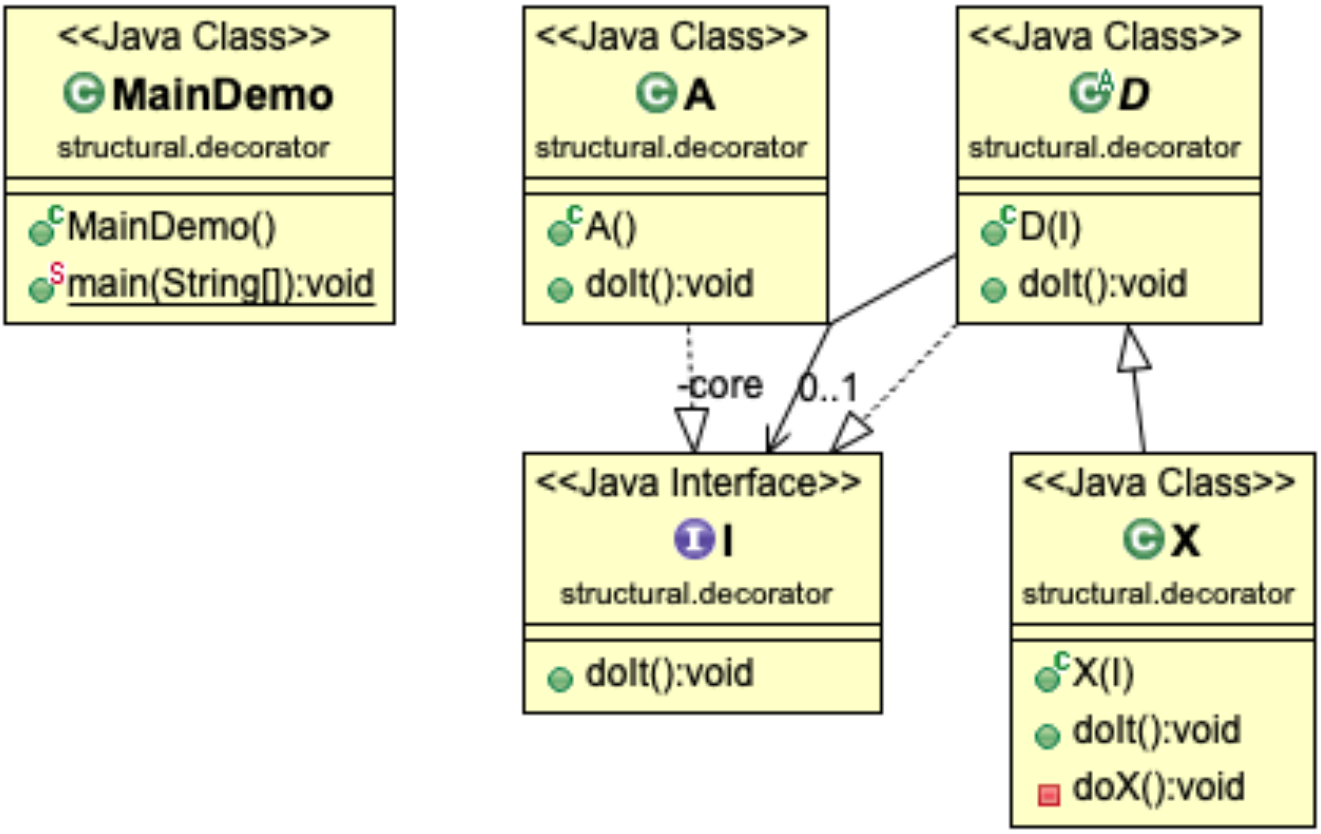

Figure 11:Class Diagram of Decorator Pattern

Output for This Code:

```
AX    AXX    AX
```

Behavioral Patterns

Behavioral design patterns are patterns that deal with encapsulating behavior with objects, assigning responsibility and managing object corporation when achieving common tasks.

This pattern concern with algorithms and the responsibilities between objects and behavioral patterns describe not just patterns of objects or classes but also patterns of communication between them.

Behavioral object patterns use object composition rather than inheritance. Some describe how a group of peer objects cooperate to perform a task that no single object can carry out by itself. An important issue here is how peer objects know about each other.

Peers could maintain explicit references to each other, but that would increase their coupling. In the extreme, every object would know about every other.

Types

- Iterator Pattern
- Template Method Pattern
- Mediator Pattern
- Observer Pattern

Iterator Pattern

This design pattern is an object behavioral pattern that provides a standardized way for accessing and traversing objects in a collection data structure.

It allows client object to access the contents of container in a sequential manner without having no knowledge about internal representation of its contents. The iterator pattern enables client object to traverse through the collection of objects.

An iterator object contains public method to allow object to navigate through list of objects within containers.

- Iterator – Defines an interface for accessing and traversing elements.
- Concrete Iterator – Implements the iterator interface and keeps track of current position in traversal of aggregate.
- Aggregate – Defines an interface for creating an iterator object.
- Concrete Aggregate – Implements the iterator creation interface to return an instance of proper concrete iterator.

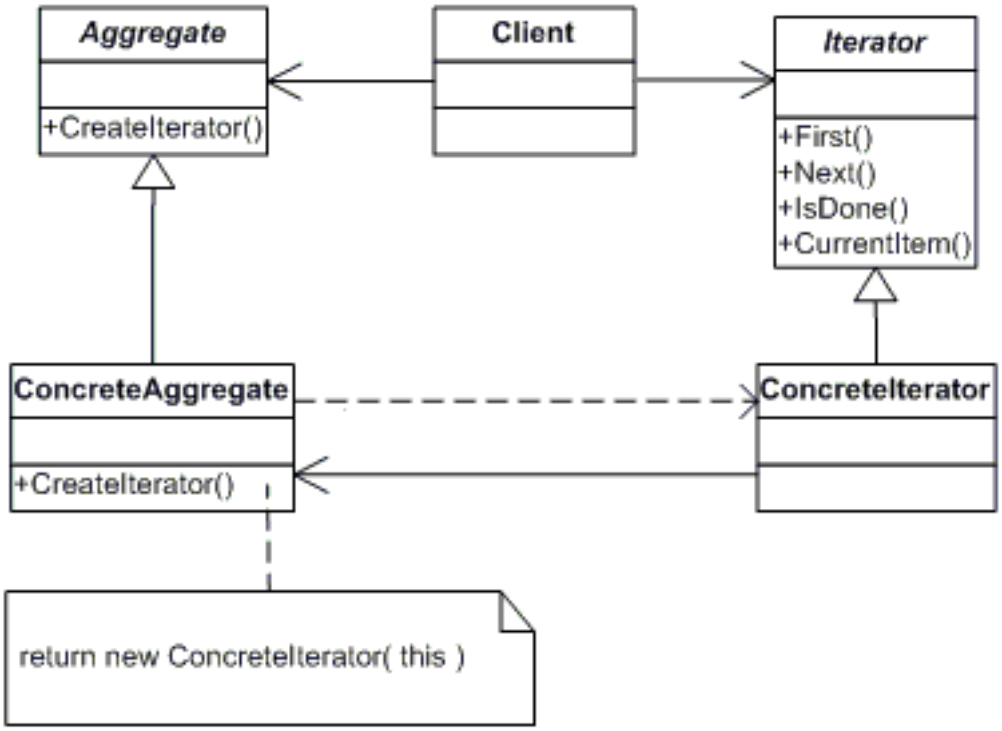

Figure 12:Iterator Pattern Example

Interface Iterator

```
package bcas.iterator;

public interface Iterator {
            public boolean hasNext();
            public Object next();
        }
```

Interface Contain

```
package bcas.iterator;

public interface Contain {
            public Iterator getIterator();
        }
```

Class NameRep Implements Contain

```
package bcas.iterator;
public class NameRep implements Contain {
            public String names[] = {"Robert" , "John"
,"Julie" , "Lora"};

            @Override
            public Iterator getIterator() {
               return new NameIterator();
            }

            private class NameIterator implements Iterator {

               int index;

               @Override
               public boolean hasNext() {

                  if(index < names.length){
                     return true;
                  }
                  return false;
               }
              @Override
              public Object next() {
                   // TODO Auto-generated method stub
                   return null;
              }
        }
}
```

Class Iterator Pattern

```
package bcas.iterator;

public class IteratorPattern {

            public static void main(String[] args) {
               NameRep namesRepository = new NameRep();

               for(Iterator iter =
namesRepository.getIterator(); iter.hasNext();){
                  String name = (String)iter.next();
                  System.out.println("Name : " + name);
               }
            }
         }
```

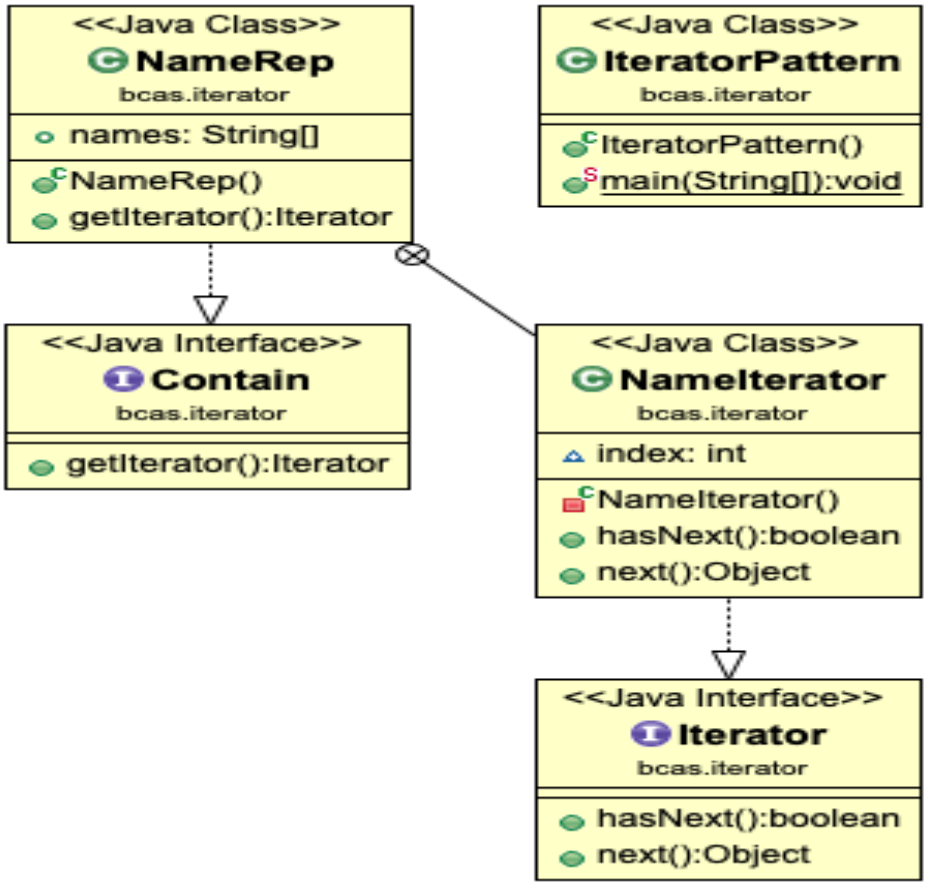

Figure 13:Class Diagram of Iterator Pattern

Output for This Code:

```
Name : Robert
Name : John
Name : Julie
Name : Lora
```

Template Method Pattern

The template pattern defines the skeleton of an algorithm in an operation. It is a behavior pattern and it will provide structure or template of an algorithm which will use by the users and users provide its implementation without changing the algorithm's structure.

The template method lets subclasses to redefine certain steps of an algorithm without changing the algorithm structure.

This pattern used in situations whether there is an algorithm and which implement multiple different ways in such scenarios the template method patterns suggest keeping the outline of algorithm in a separate method and it is referred as template method in a class.

- **Where Template Design Pattern Uses:**
    - Implement in invariant parts of an algorithm once and leave it up to subclasses to implement the behavior that can vary.
    - Common behavior among subclasses should be factored and localized in common class to avoid code duplications.

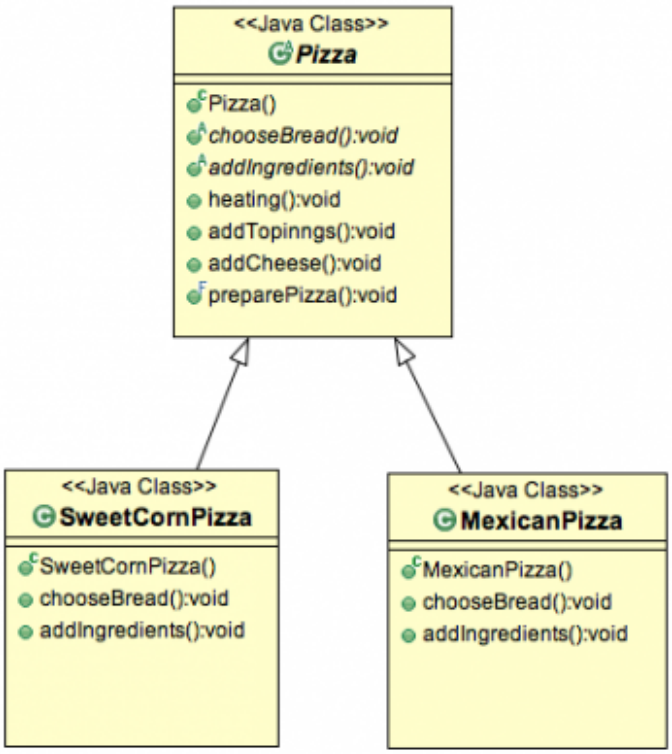

Figure 14:Example of Template method pattern

Abstract Class Game

```
package bcas.template;

public abstract class Game {
     abstract void initialize();

     abstract void startPlay();

     abstract void endPlay();

     public final void play() {
          initialize();
          startPlay();
          endPlay();
     }
}
```

Class FootBall Extends Game

```
package bcas.template;

public class FootBall extends Game {

             @Override
             void endPlay() {
                System.out.println("Football Game Finished!");
             }

             @Override
             void initialize() {
                System.out.println("Football Game Initialized!
Start playing.");
             }

             @Override
             void startPlay() {
                System.out.println("Football Game Started.
Enjoy the game!");
             }
          }
```

Class Cricket Extends Game

```
package bcas.template;

public class Cricket extends Game {

            @Override
            void endPlay() {
               System.out.println("Cricket Game Finished!");
            }

            @Override
            void initialize() {
               System.out.println("Cricket Game Initialized!
Start playing.");
            }

            @Override
            void startPlay() {
               System.out.println("Cricket Game Started.
Enjoy the game!");
            }
}
```

Class TemplatePattern With Main Method

```
package bcas.template;

public class TemplatePattern {
     public static void main(String[] args) {

          Game game = new Cricket();
          game.play();
          System.out.println();
          game = new FootBall();
          game.play();
     }
}
```

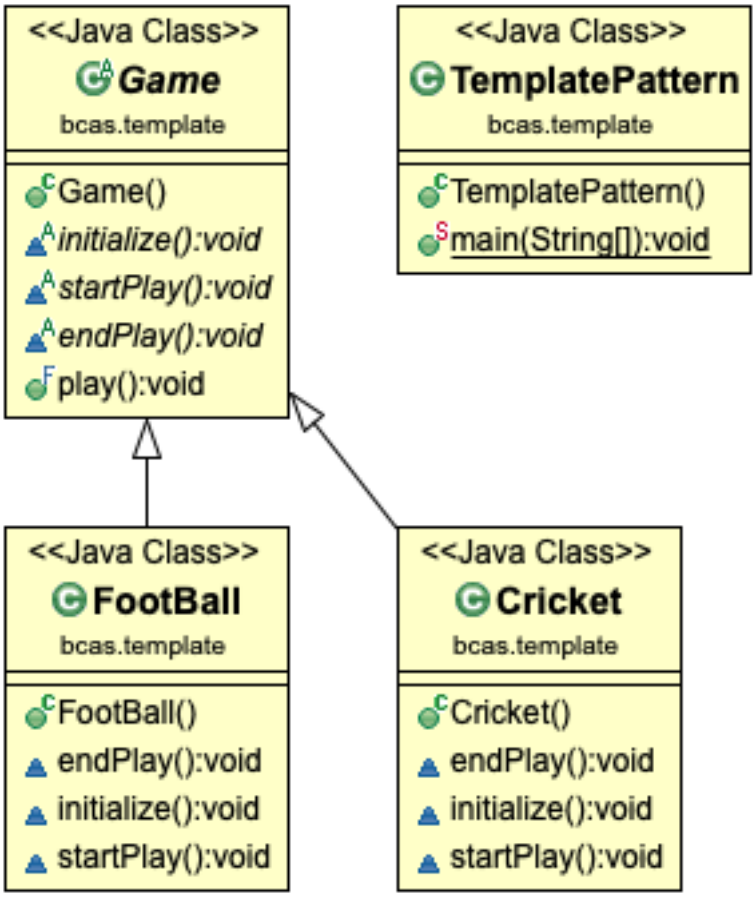

Figure 15:Class for the Given Template method pattern

Output for This Code:

```
Cricket Game Initialized! Start playing.
Cricket Game Started. Enjoy the game!
Cricket Game Finished!

Football Game Initialized! Start playing.
Football Game Started. Enjoy the game!
Football Game Finished!
```

Part 1.3: Relationship Between Object Oriented Paradigm and Design Patterns.

Design patterns are based on practical solutions that have been successfully implemented over and over again. Design patterns represent a means of transition from analysis/design to design/implementation. A design pattern is an abstraction from a concrete recurring solution that solves a problem in a certain context

The main relationship between object-oriented paradigms and Design patterns are if we want to use oop in programming for the effective code we will definitely use design patterns. Design patterns helps to implement the characteristics of the object-oriented paradigm.

In the software development the effective and correction of code is very important for that purpose we use design patterns to implement the codes in a good and effective manner. Patterns give solution to the problems in easier manner

Design patterns are nothing but a solution to a recurring problem and the solution is robust, scalable, readable and flexible. They can exist for other programming paradigms too

L02

L02: Design a Series of UML class Diagrams

Part 2.1: Design A Class Diagram for The Scenario.

UML (Unified Modeling Language) is a graphical notation for drawing diagrams of software concepts.

UML has three main types of diagrams.

- Static Diagrams – Describes unchanging logical structure of software elements by depicting class, objects and data structures and relationship that exist between it.
- Dynamic Diagrams – Show that how software entities change during execution by depicting flow of execution or way entities change state.
- Physical Diagrams – Show that unchanging physical structure of software entities by depicting physical entities such as source files, libraries, binary files and relationship exist between them.

UML is enormously helping for communicating design concepts between software developers and engineers. Lot can be done with small group of developers with white board and if have some ideas then need to communicate with others, so that which can be a big benefit.

A UML class Contains Attributes and Operations.

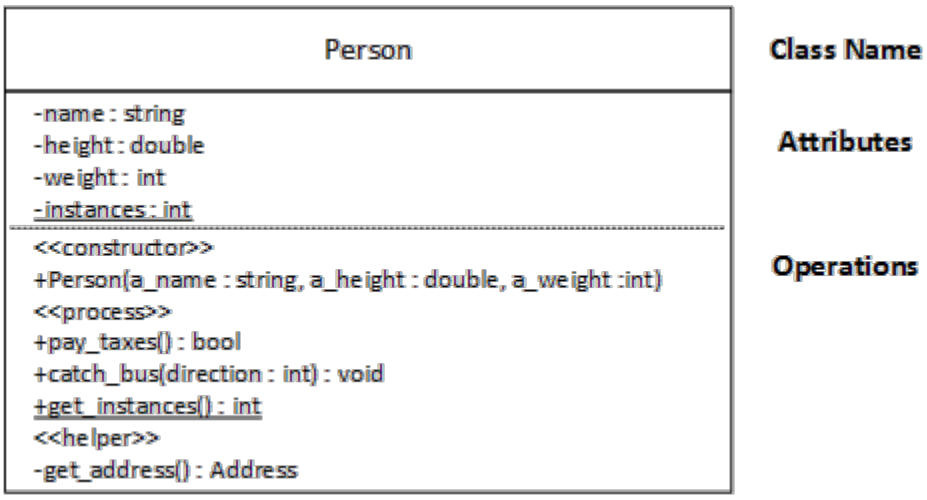

Figure 16:UML diaram attributes and Operations

Class diagrams are perhaps one of the most common UML diagrams used and class diagram symbols center around defining attributes of a class. These are some notations of UML class diagrams

Figure 17:Symbols of UML diagram

Here below are some line notations used in UML class Diagrams.

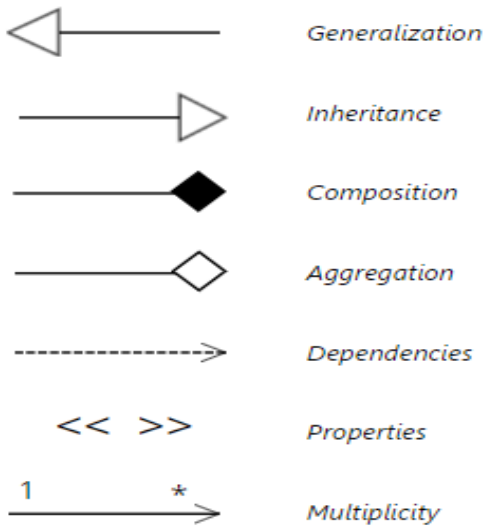

Figure 18:Line notations for UML diagrams

Complex applications with many collaborators will require a solid function of planning and clear, concise communication among team members as project progress and visualizing user interactions, processes and structure of system so these purposes we want to use the UML diagrams.

According to the scenario the Class Diagram as shown below:

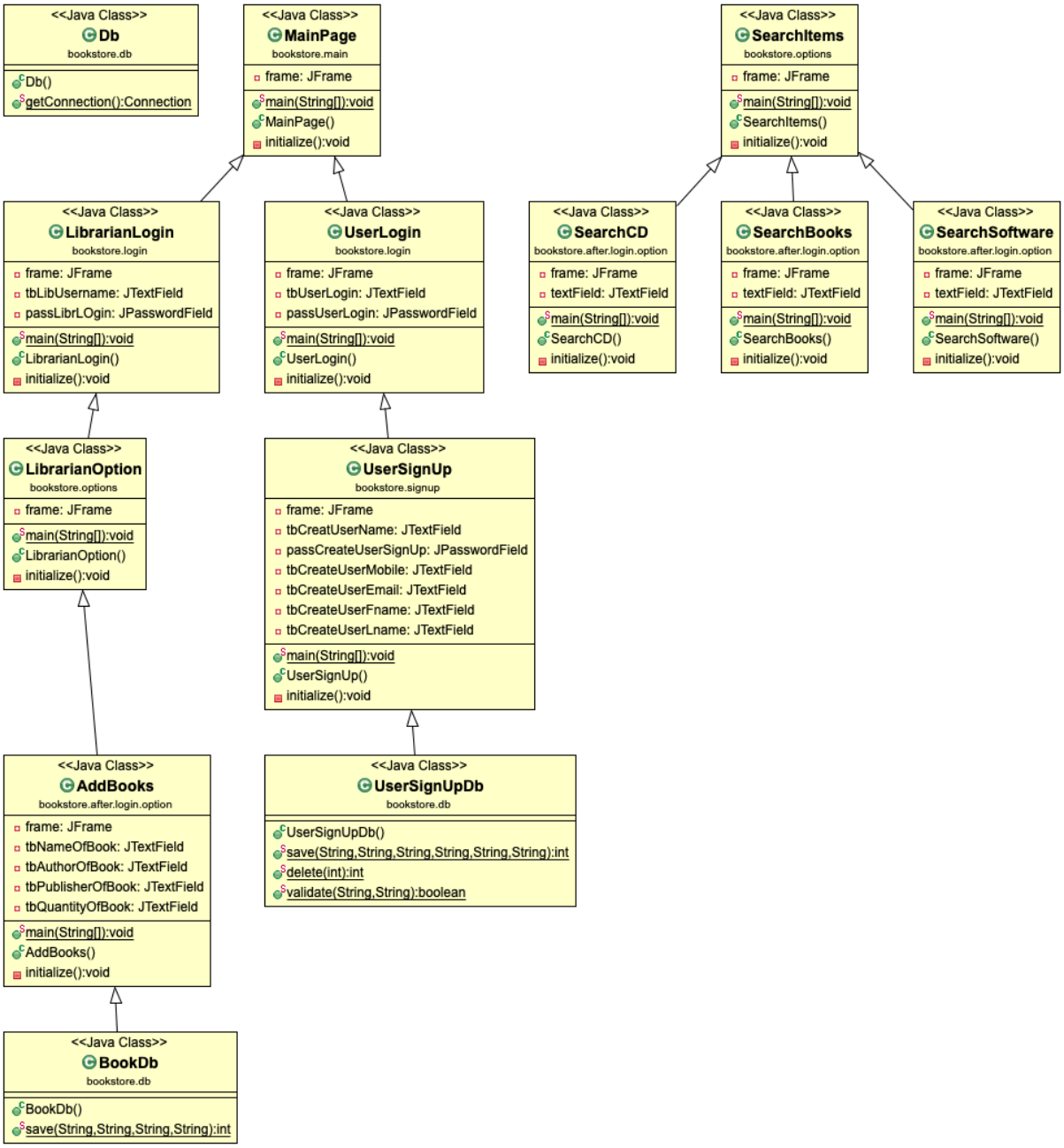

Figure 19:UML diagram of Given Book Store Scenario

This is the scenario-based Class Diagram for Book Store System and in this We have 2 that is Librarian and Users. Librarian able to add Books and View books and will delete books.

Users able to sign up and login after that they will have a main page that including three options like Search for books, Search for CD and Search for Software's. After that they will add to cart and View their cart information and they able to pay for their orders. This is the scenario based on the class diagram.

Part 2.2: Discuss What Type of Test Driven Development

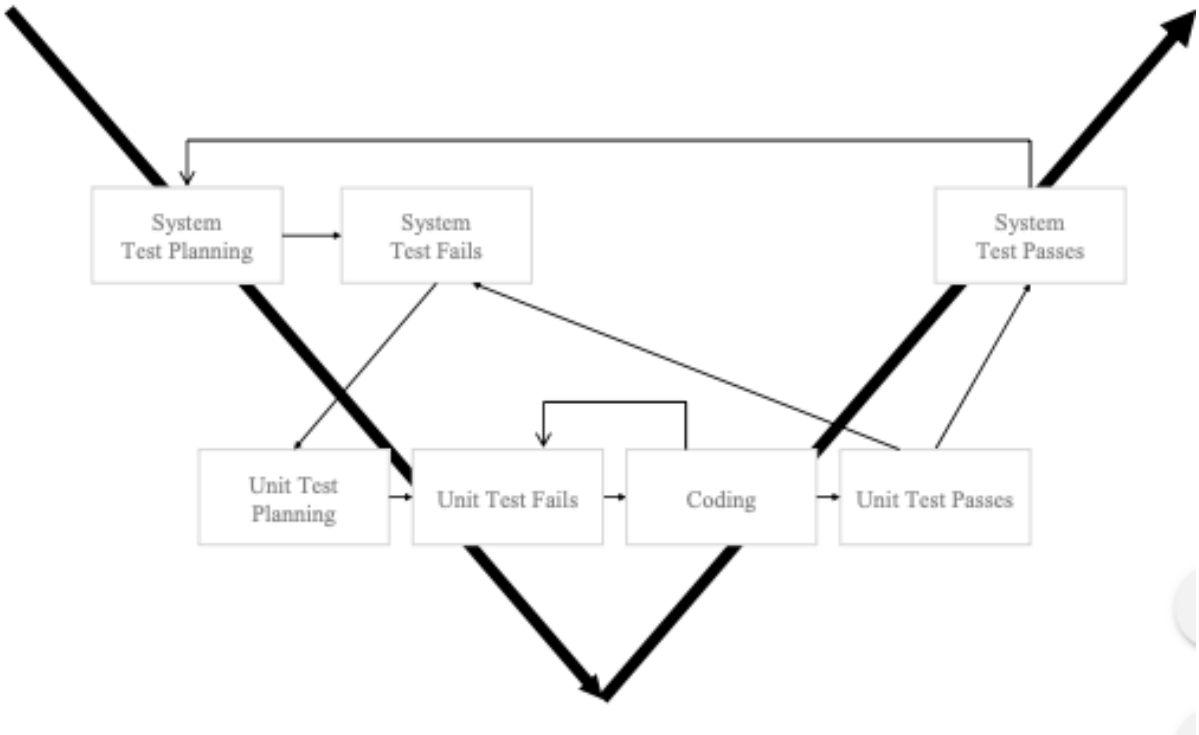

Figure 20: Test Driven Development Process

Testing of software is the effective method to evaluate and improve the quality of the software. The Test-Driven development is considered to be the one of the best methods. It focusses on unit testing where the developers incrementally write the tests before any code fragment and is considered to be effective development approach.

In the software development Code quality is the important factor and long-term high-quality code is more useful and maintainable that low quality code. TDD reduces the complexity if used persistent over time.

Unit Testing and TDD are not same and Unit testing verifies a behavior of unit separately from other units of code. With Unit Testing developers will test the single units of their code.

But in TDD all tests are written before all the codes and In TDD the unit tests are write before any code fragment written.

Advantages

- Improve of Code Quality and Code Design
- Easier and Fast to make changes
- Easy to add new features
- Reduce Maintenance
- Code Documentation

Disadvantages

- Complicated and badly written unit tests which can result in extremely fragile tests.

There are two levels in TDD and they are Acceptance TDD and Developers TDD and with Developer TDD write a single developer test and sometimes referred to as unit test and then enough code to fulfill the test and the goal of that is to specify a detailed executable design for the solution on JIT (Just in Time) basis.

The testing in 3 ways such as one test at a time, building up the functionality of system and write the test first.

The purpose of TDD is to enable small steps when writing software's and this promoted for long term because it is do far more productive that attempting to code in large steps.

Tools

- Junit – The developer able to check business logic of any class.

- JWalk – It supports a testing paradigm called Lazy Systematic Unit Testing.

- TestNG – It covers test categories such as unit, functional, end-to-end, integration.

- JTest - Unit test-case generation and execution, static analysis, regression testing, code coverage, and runtime error detection.

L03

L03: Implement Code Applying Design Patterns

Part 3.1: Develop Code That Implements Design Pattern.

For the given scenario I Developed code using Windows Builder Jframe Tools and for every changes I did on code are committed through the new Code development and monitoring application Git Lab.

Here below I submit the GitLab link of my development code:

https://gitlab.com/nirojan2206/oxford-bookstore-system-

```
import java.awt.EventQueue;
import javax.swing.JFrame;

public class Sample {

	private JFrame frame;

	public static void main(String[] args) {
		EventQueue.invokeLater(new Runnable() {
			public void run() {
				try {
					Sample window = new Sample();
					window.frame.setVisible(true);
				} catch (Exception e) {
					e.printStackTrace();
				}
			}
		});
	}
	public Sample() {
		initialize();
	}

	/**
	 * Initialize the contents of the frame.
	 */
	private void initialize() {
		frame = new JFrame();
		frame.setBounds(100, 100, 450, 300);
		frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
	}

}
```

Here I put a sample code of Jframe builder when in the initialization portion and this code development done in Factory Design Pattern. The complete code will go through the link of GitLab.

Part 3.2: Test Case for The Source Code.

In this we will see about the test cases and their result mainly using unit testing to test the developed code for Book Store according to the scenario.

Test Cases	User and Admin Input	Result	Criteria
User Registration	User input already selected username	Passed	Displays Username already exist
User Registration	User do not fill the textbox	Passed	Displays please fill the empty field
User Login	User Only Fill Username and click login button	Passed	Displays Please Enter the Password
User Login	User Not enter any data and press login button	Passed	Display Please enter user name and password
User Registration	User Enter all the details	Passed	Displays that your Account Created
User Login	User enters Wrong Password	Passed	Displays Incorrect password
User Login	User enters correct username and password	Passed	Displays Login Successfully
Add To Cart	User select a CD and click add to cart button	Passed	Displays Your item successfully added to cart
Add Books	Librarian create book	Passed	Book added successfully
Add To Cart	User view the cart information	Passed	Cart information of the specific user viewed
User Login	User will enter valid user name but wrong password and click submit.	Passed	Please enter the Correct password.

L04

L04: Investigate scenarios with Respect to Design Patterns.

Part 4.1: Design Pattern for The Scenario

For the scenario of the Book Store the builder pattern is suitable for that and builder pattern separates the construction of a complex object from its representation so that the same construction process can create different representations.

Why I Choose:

This pattern is used by many book stores to produce many options. Book Store mainly contains main classes, sub classes, the construction process is same and when a customer add book in to cart and the librarian will assemble and check their items the they will be the order place and ready for shipping.

Some times creational patterns are complementary and builder can use of the other patters to implement which the components getting built. Builder will build a composite and design start out using factory method is less complicated and more customizable.

Evolve toward abstract factory, prototype or builder is more flexible as designer discovers whether that more flexible. It avoids confusion because user will decide for which will set values and which is not.

Conclusion

Really, I am so happy in completing this assignment because I have learned a lot by doing this assignment and also, whatever the things I missed to understand in the lecturers now with the help of assignment I have understood. And also, now I have the confident in developing the software system an also I have the experience in developing a software system. Mainly I have analysis about test cases for the source code and it was wonderful because they found me some mistakes on my assignment. It was good experience.

References

Alexander, 2010. *Git Hub.* [Online] Available at: https://github.com/dieuph/ [Accessed 10 03 2019].

Anon., 2014. *web.cs.ucla.edu.* [Online] Available at: http://web.cs.ucla.edu/~palsberg/paper/toplas06.pdf [Accessed 15 02 2019].

Anon., 2015. *xenonstack.* [Online] Available at: https://www.xenonstack.com/insights/what-is-test-driven-development [Accessed 20 03 2019].

Anon., 2017. *javapapers.com.* [Online] Available at: https://javapapers.com/oops/association-aggregation-composition-abstraction-generalization-realization-dependency/ [Accessed 18 03 2019].

Jenkov, J., 2016. *jenkov.com.* [Online] Available at: http://tutorials.jenkov.com/java-unit-testing/simple-test.html [Accessed 21 03 2019].

MartinFowler, 2015. [Online] Available at: https://martinfowler.com/bliki/SelfTestingCode.html [Accessed 18 03 2019].

Noble, J., 2019. *homepages.ecs.* [Online] Available at: http://homepages.ecs.vuw.ac.nz/~kjx/papers/classify.pdf [Accessed 19 02 2019].